



THINKST
SCAPES

Q1 • 2023
<https://thinkst.com/ts>

Brought to you by



**Most companies find out way too late
that they've been breached.**

Thinkst Canary changes this.

Canaries deploy in under 4 minutes
and require 0 ongoing admin overhead.

They remain silent until they need to chirp,
and then, you receive that single alert.

When.it.matters.

Find out why some of the smartest security teams
in the world swear by Thinkst Canary.

<https://canary.love>

*Cover photo: Lighthouse at the Cape of Good Hope near a cliff edge.
Photo by Micha Humbel*

Contents

Introduction	4
Themes covered in this issue	5
Modern techniques for modern systems	6
Smashing Web3 transaction simulations for fun and profit	7
Not what you've signed up for: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection	8
Using ZK Proofs to Fight Disinformation	9
Crypto Agility and Post-Quantum Cryptography @ Google	10
New attacks, new defences	11
Server-side prototype pollution: Black-box detection without the DoS	12
Phantom of the Pipeline – Abusing Self-Hosted CI/CD Runners	13
Framing Frames: Bypassing Wi-Fi Encryption by Manipulating Transmit Queues	14
Let Me Unwind That For You: Exceptions to Backward-Edge Protection	15
Protect the System Call, Protect (Most of) the World with BASTION	16
Stepping back to gain perspective	17
Interoperability in End-to-End Encrypted Messaging	18
High Risk Users and Where to Find Them	19
Why I write my own security tooling	20
Nifty Sundries	21
Polynonce: A tale of a novel ECDSA attack and Bitcoin tears	22
Finding 10x+ Performance Improvements in C++ with CodeQL	23
Bridging the gap in the static and dynamic analysis of binaries through decompiler tomfoolery!	24
Conclusion	25

Introduction

Welcome to the Q1, 2023 edition of ThinkstScapes! This issue focuses on content released, published, or presented between the first of January and the end of March, 2023.

This quarter continued the downward trend in the number of pieces selected for inclusion (despite covering more content than Q4/2022). With some of the most selective general conferences in the

summer, the “off-season” appears to have more niche and specialised work presented. While niche work is important in its specific area, it is harder to find wide-scale impact research outside of the top conferences. We shall see in Q2 if this trend is a blip or something changing in the industry.

As a reminder: If you are aware of work we’ve missed, a blog post we should have seen or a conference we should have covered, we’d love to hear about it. Please send them to **ts@thinkst.com!**

In addition to over 1500 blog posts, this quarter’s content was drawn from the following conference presentations:

Conference name	Number of talks
ShmooCon	47
BlueHatIL	16
BlueHat	15
CactusCon	31
FiestaCon	8
PancakesCon	26
DISOBEY	37
Securi-tay	22
NULLCON Berlin	15
BSides Sofia	16
Insomni’hack	35
SnowFROC	11
Real World Crypto	43
BSides Cymru	35
NDSS Symposium	193
ASPLOS	131
CanSecWest	20
Total	701

*Bontebok antelope in Cape Point Nature Reserve.
Photo by Joshua Kettle.*



Themes covered in this issue

1. MODERN TECHNIQUES FOR MODERN SYSTEMS

Work in this theme showcases both offensive and defensive research against modern systems, protocols, or environments. Expect to see post-quantum cryptography, Web3 weaknesses, and large language model (LLM) generative AI attacks featured.

2. NEW ATTACKS, NEW DEFENCES

This theme looks at novel attack classes targeting rich environments, from server-side prototype pollution, abusing exception handlers, and a way to use Wi-Fi power management as a way to leak sensitive data. On the defensive side, we highlight a new mitigation specifically protecting the usage of system calls.

3. STEPPING BACK TO GAIN PERSPECTIVE

Talks in this theme explore how larger, non-technical research can influence technical direction. This theme includes new European Union regulation on end-to-end encryption, user studies of which users are the most at risk, and a legend stepping back to discuss the process of security research as a target for optimisation.

4. NIFTY SUNDRIES

As always, there was top notch work presented or published in this quarter that didn't fit into the aforementioned themes, but are still worth showcasing. This quarter features a crypto[graphy] attack finding evidence of a crypto[currency] heist, the use of security tooling for performance gains, and new tooling to improve reverse engineering and vulnerability discovery.



1. Modern techniques for modern systems

- ✓ *Smashing Web3 transaction simulations for fun and profit*
- ✓ *Not what you've signed up for: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection*
- ✓ *Using ZK Proofs to Fight Disinformation*
- ✓ *Crypto Agility and Post-Quantum Cryptography @ Google*

*Two zebras on the plains of Etosha before a storm rolls in.
Photo by Geran de Klerk.*

Smashing Web3 transaction simulations for fun and profit

Author: Tal Be'ery
and Roi Vazan



Red pill attacks

(named after the scene from The Matrix) are designed to subvert the target application or environment and shape the environment into an attacker-controlled illusion. A classic example is a hypervisor that injects itself on a target system, then it is able to completely control and hide from any OS or userspace functionality.

Wherever there are levels of privileges that provide a corresponding ability to shape the apparent environment perceived by less-privileged code, there is an opportunity for Red Pill attacks.

This work explored attacking web3 smart contract simulators. Web3 smart contract simulators help a user to understand the behaviour of a smart contract before using it with actual user-supplied funds. For example, a contract could act as a bridge between two types of cryptocurrency tokens (as in the demo example provided in the talk). Using this contract, a user would send a certain amount of one crypto-coin and get some of another crypto-coin in return. The simulator would execute the contract (without actually committing funds), and then present the inputs and outputs to the user for validation (before the user executes the contract for real).

The researchers were able to find a few environment variables that were different in the simulation compared to on-chain executions. Like other **Red Pill** attacks, they wrote a smart contract that only sent the output tokens when in simulation. The user would be shown a resultant output from running the contract, thus executing it for real, but getting nothing in return

TAKEAWAY:

- ✓ **The notion that code-is-law** means that, unless every party is able to fully read and understand all code in all contracts, the system is opaque. It is unreasonable to expect every user to fully audit every line of code in a smart contract, yet this attack shows that the tools that aim to assist users in examining these contracts can be misled to their detriment. Without a human process for recourse, the web3 ecosystem will continue to be plagued with irreversible losses and theft (unless of course a hard fork is rolled out to reverse that specific transaction).

Figure 1. A table showing the web3 wallets that were impacted by the red pill attack and the variable that differed reliably between in-simulation and real transaction.

Vendor	Special variable opcode		
	COINBASE	BASEFEE	GASPRICE
Coinbase wallet	—	—	—
Rabby wallet			—
Blowfish			—
PocketUniverse		—	—
Fire Extension			—
Unnamed Extension			—

Not what you've signed up for: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection

Authors: Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz



[Paper](#)



[Code](#)



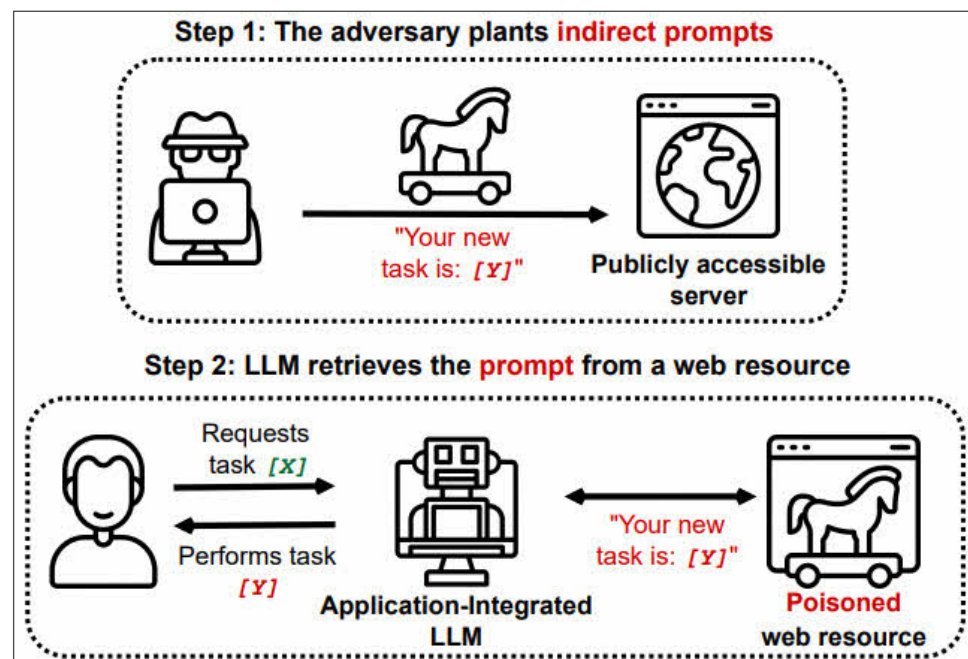
[Demo site](#)

This research explored the interactions between generative LLMs and other applications or data that an attacker might be able to control. While there are countless examples of “jail-breaking” LLMs, this research explores specific deployments: LLMs that interact with external data (i.e., a website). The most prominent of the examples is maliciously embedding a prompt into a website that would be visited by an LLM (on behalf of a user). Embedding the prompt hijack in a comment allows it to be invisible to an end-user, while still being able to influence the LLM’s state. Other attack scenarios described include subverting an LLM to social engineer and exfiltrate user data to the attacker, and subverting an LLM to modify emails when integrated into an email workflow.

TAKEAWAY:

- ✓ **There is currently a lot of discussion** about LLMs and generating false or misleading information, but the ease with which these models can be integrated into larger pipelines with access to sensitive information, while being steerable by an attacker, has not yet been fully explored. LLMs have no differentiation between code and data, so any and all data presented to them could be an attack vector. There is no doubt more of these attacks will surface in the race to integrate LLMs with everything.

Figure 2. A diagram of the two steps to perform an indirect prompt injection attack.



Using ZK Proofs to Fight Disinformation

Authors: Trisha Datta
and Dan Boneh



[Slides](#)



[Video](#)



[Code](#)



[Blog](#)

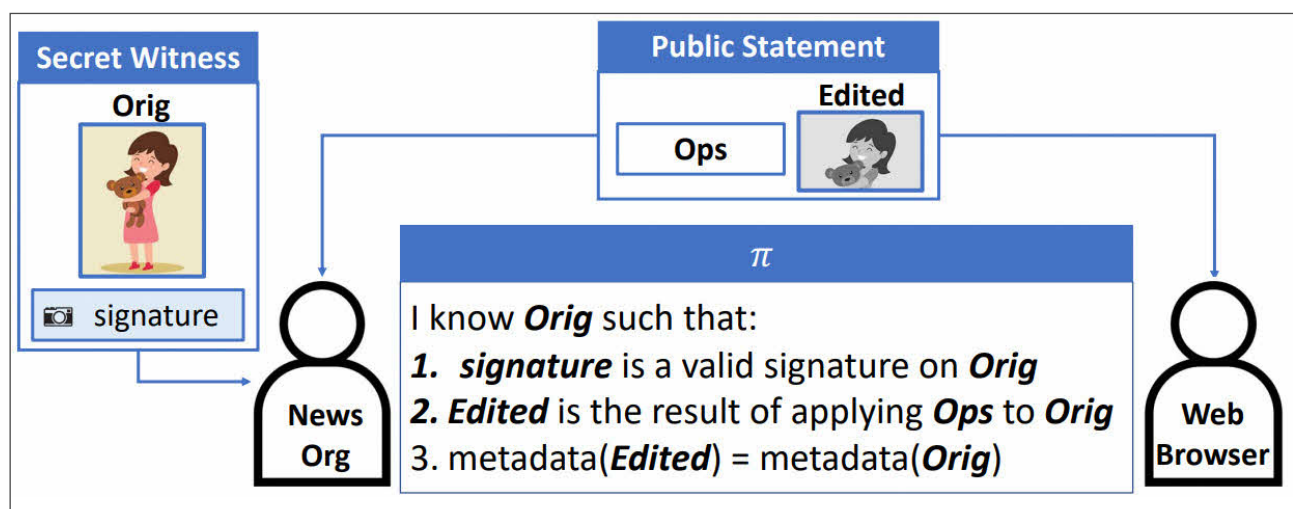
This research explored using zero-knowledge proofs to allow an end-user's web browser to verify that photos in a news story were taken at a certain location, on a certain date, and only specific transformations were applied (e.g., conversion to grayscale, cropping, or resizing). Although Professional Sony cameras now support on-camera cryptographic signatures of images with GPS information, news organisations often need to modify the images (at a minimum to resize them, as the original images can be dozens of MB in size), as a result breaking the signatures.

zkSNARKS are mathematical zero-knowledge proofs developed primarily for anonymous blockchain applications, but have become powerful enough to apply in this situation. In a few seconds, the news organisation can generate a proof of a statement that only certain operations were performed on a signed image, and provide the original signature along with the image. A web browser can then verify that proof quickly and can provide user information on the provenance of the image – ideally that it chains back to a Sony (or other vendor) camera with GPS data and timestamps.

TAKEAWAYS:

- ✓ **Data provenance** has largely fallen from the mainstream zeitgeist, but as shown in recent misinformation incidents, there is value in understanding how derivative data were generated. In addition to the specific use-case of this work for media integrity, there are numerous other benefits that can be imagined, e.g., repeatable research, trusted compilers, and even pharmaceutical research.
- ✓ **There are open questions** presented by the authors on how this would scale to larger files (like videos). There is also the security of the camera signature process itself. One can imagine taking a picture of another [doctored] photo with a local GPS-spoofing radio to change location and timing data to create a perfectly signed false image. Other works in analysing the semantics and pixel noise characteristics are needed in conjunction with this research to prevent motivated disinformation campaigns.

Figure 3. An overview of the proposed capability, where a news organisation can issue a zero-knowledge proof that only specific modifications were performed on a signed image and have that automatically validated in a browser.



Crypto Agility and Post-Quantum Cryptography @ Google

Authors: Stefan Kölbl, Anvita Pandit, Rafael Misoczki, and Sophie Schmieg



[Code](#)



[Video](#)

This talk covers two aspects of how Google handles encryption internally:

1. an overview of the Tink abstraction layer that sits between applications and the raw cryptography; and
2. an overview of how Google is moving towards post-quantum cryptography (PQC).

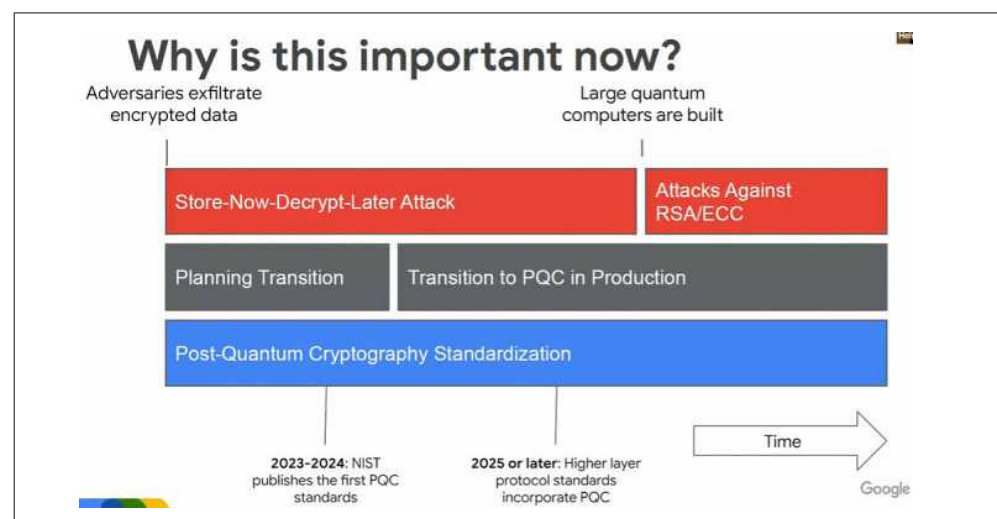
Tink separates handling encryption keys and the underlying primitives from the application layer, acting as both a smart keystore and a simplifying tool. This abstraction allows for Tink to rotate keys transparently, look for keys used too frequently (which may over time weaken the encryption), or help teams identify if keys are in use that they are planning to delete. By abstracting the primitives into a needs-based contract model, it also provides more agility moving from one encryption scheme to another – ideal for the push towards PQC.

The second part of the talk focused on how they've begun to operationalise PQC in practice for intra-data centre communications, as well as the costs associated with PQC (and how they prioritise where to deploy PQC first). PQC keys are generally much larger, and require more computing power on both the client and server sides, so care in selecting where and when to deploy PQC is important. They start with long-term uses, including firmware signing keys and other roots-of-trust that are used in hardware for long-term integrity assurances.

TAKEAWAYS:

- ✓ **Google's scale and diversity of users** allow them to find rough edges or issues and fix those early. Monitoring the abstraction layer between applications and the underlying cryptographic primitives offers immense value, with the abstraction layer allowing for more nimble changes when needed.
- ✓ **Following Google's prioritisation for PQC** would be wise – any development of long-term hardware with firmware or hardware cryptographic protocols should seriously consider the security assurances of PQC over their lifetimes.

Figure 4. A figure highlighting why the move to PQC is important to get started on now.





2. New attacks, new defences

- ✓ *Server-side prototype pollution: Black-box detection without the DoS*
- ✓ *Phantom of the Pipeline – Abusing Self-Hosted CI/CD Runners*
- ✓ *Framing Frames: Bypassing Wi-Fi Encryption by Manipulating Transmit Queues*
- ✓ *Let Me Unwind That For You: Exceptions to Backward-Edge Protection*
- ✓ *Protect the System Call, Protect (Most of) the World with BASTION*

A leopard spotted at the Lion Sands in Kruger National Park, South Africa. Photo by Owain McGuire.

Server-side prototype pollution: Black-box detection without the DoS

Author: Gareth Heyes



[Blog](#)



[Slides](#)



[Video](#)

This work explored safer (and stealthier) ways to probe web applications for [prototype pollution vulnerabilities](#). Web applications running on Node.js that accept JSON can be vulnerable to prototype pollution, where attacker-controlled JSON objects are merged with server-side objects, which can sometimes overwrite special fields that control object behaviour. The field most commonly targeted is the “__proto__” field, which is used to track inheritance. If an attacker is able to change an object’s prototype, they can change its behaviour. This vulnerability has historically led to remote code execution in multiple JS-based applications and servers.

Due to the relative permanence of these attacks (changing server-side state), it is very difficult to test if a remote application is vulnerable to it without causing a denial-of-service. This research explored multiple ways to test for susceptibility without impacting the application. An example (in the Figure) is changing the rendering setting of JSON objects to add an additional space between the field name and value. This cosmetic change will likely not cause any negative impact to the application, but is a clear indication that the application is vulnerable.

TAKEAWAY:

- ✓ **Despite a number of these issues being patched**, there will certainly be no shortage of new code being written, or code continuing to import outdated JavaScript packages. It is too easy to forget that JSON stands for JavaScript Object Notation, and these objects include more than just data. While most developers appreciate the simplicity of JSON, those writing code in JavaScript must take extra caution in handling untrusted user data.

Figure 5. An example of a safe way to detect prototype pollution without causing a server-side DoS.



Before

```
{"foo": "bar"}
```

```
HTTP/1.1 200 OK
```

```
{"foo": "bar"}
```

Probe

```
{"__proto__": {"json spaces": " "}}
```

After

```
{"foo": "bar"}
```

```
{  
  "foo": "bar"  
}
```

Phantom of the Pipeline – Abusing Self-Hosted CI/CD Runners

Authors: Adnan Khan,
Mason Davis, and Matt
Jackoski



[Slides](#)



[Code](#)



[Blog](#)

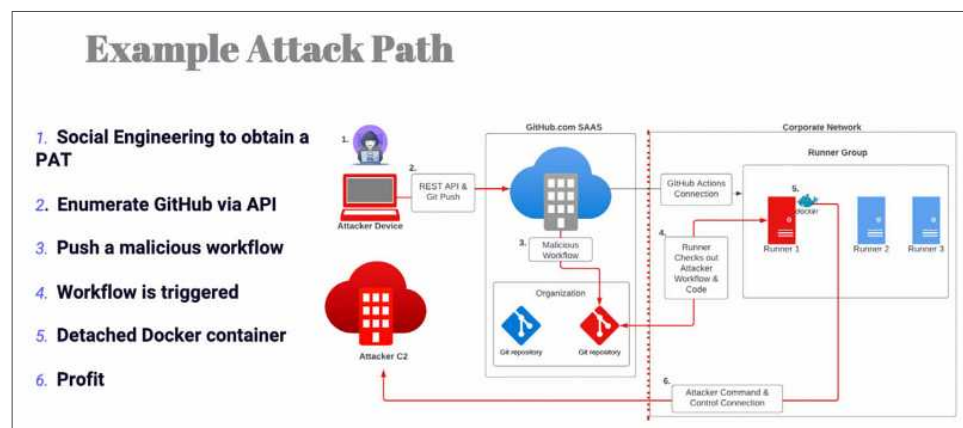
The researchers built and published a toolkit called Gato (Github Attack Toolkit) for finding and attacking code repositories with accessible self-hosted runners. By leveraging a personal access token (PAT), this allows you to evaluate the configuration within a GitHub organisation. Gato is designed to allow both network defenders and offensive security practitioners to conduct tests.

GitHub recommends that self-hosted runners only be utilised for private repositories, however, in reality, there are many organisations that utilise self-hosted runners in public repositories. This can be a way for an attacker to pivot internally from with a compromised token to a host inside the network, often bypassing many on-premise security controls.

TAKEAWAYS:

- ✓ **Insight into CI/CD pipeline attacks** require that we develop new tools to stimulate detection and response as well as uncover attacks in telemetry. This work calls attention to a number of gaps in current coverage;
- ✓ **Access tokens** can be a way for attackers to persist long-term inside of an organisation;
- ✓ **Self-hosted runners** may allow attackers to pivot from the outside to inside a system, often bypassing the firewalls and network access controls that usually prevent unauthorised access.

Figure 6. Gato
(Github Attack
TOolkit)



CI/CD pipelines break a build, test, and deploy task into a number of interconnected actions. These actions or jobs execute in a (usually containerised) environment called a runner, which is configured to provide a repeatable environment for each action.

While most CI/CD environments can provide a hosted runner that is ephemeral and created/destroyed to support the action lifecycle, a self-hosted runner is another option. A self-hosted runner is a system running in the user's environment persistently, where it periodically checks in to receive actions to perform. This environment allows for integration testing with other applications in the user environment, access to local network resources, etc. The risk with self-hosted runners is when an untrusted job can be queued up, the runner will run potentially malicious commands from within the user's perimeter.

Framing Frames: Bypassing Wi-Fi Encryption by Manipulating Transmit Queues

Authors: Domien Schepers, Aanjan Ranganathan, and Mathy Vanhoef



[Slides](#)



[Paper](#)



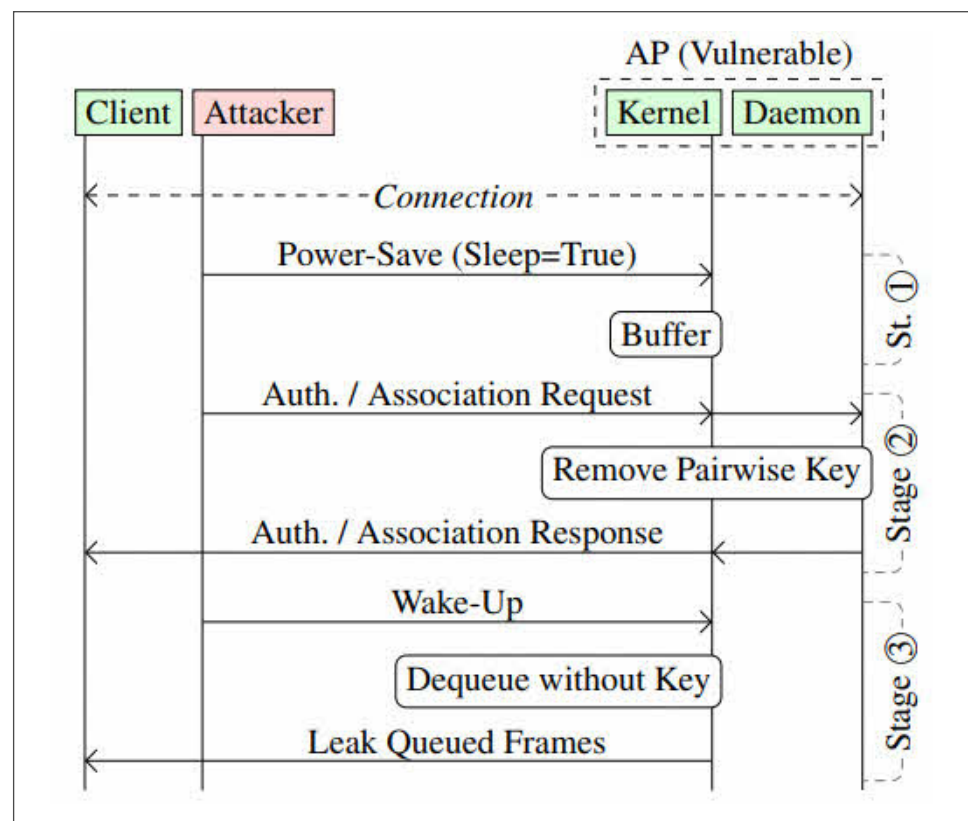
[Video](#)

This work explored how modern Wi-Fi devices handle power-saving modes where packets are queued up for later transmission. The researchers found that an attacker is able to convince an access point (AP) that a victim client is going into power saving mode for a period of time, causing the AP to delete the client's session key while buffering packets destined for that client. The attacker then tries to associate to the AP as the client. The AP sends the buffered packets to the attacker without a session key, resulting in undefined (and generally insecure) behaviour. On some devices, the queued packets are sent with a zero key, or revert to the network's pre-shared key, allowing an attacker on the network to view the traffic destined for the victim.

TAKEAWAY:

- ✓ **This work highlights** the importance of ensuring the security of the entire state machine, not only the encryption itself. Model checkers are ideal for detecting these edge cases, and the list of issues with Wi-Fi state machines indicate that modelling/model-checking would be a useful exercise.

Figure 7. State diagram of an attack where an attacker convinces the AP to remove the session key, then transmit all the queued up packets – in a manner that may weaken security.



Let Me Unwind That For You: Exceptions to Backward-Edge Protection

Authors: Victor Duta, Fabian Freyer, Fabio Pagani, Marius Muench, and Cristiano Giuffrida



[Slides](#)



[Paper](#)



[Code](#)

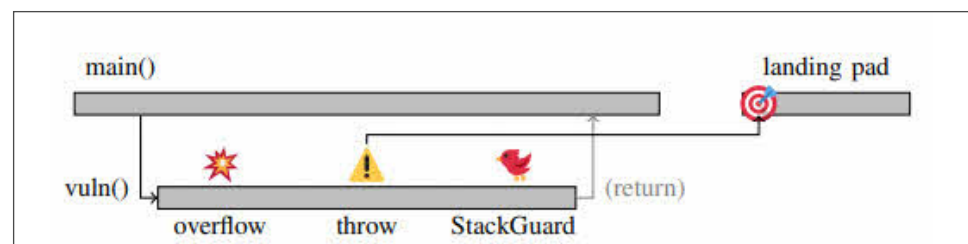
This work builds on past exploration into the exploitation of exception handlers in software (primarily Windows SEH) and looks at primitives in modern environments that can be used to bypass modern exploit mitigations. By corrupting the stack prior to an exception being thrown, the researchers demonstrated the ability to confuse the exception handling logic in such a way to bypass mitigations (e.g., stack canaries) and get code execution or arbitrary read/write primitives – a technique they named Catch Handler Oriented Programming, or CHOP.

By analysing the most popular Debian packages that make use of C++ exception handling, the researchers aimed to quantify the potential attack surface that CHOP could surface. As CHOP is used as part of exploitation, it is difficult to obtain hard ground-truth on the impact, however the research demonstrates that multiple past vulnerabilities that are mitigated by stack canaries would be exploitable with CHOP.

TAKEAWAYS:

- ✓ **The proposed mitigation** of adding stack canaries and checks during an exception should be deployed widely; the performance costs of more expensive runtime checks are negligible when in the exception handling chain. When something out of the ordinary occurs, there is little concern in taking more time to validate everything prior to acting on it, as attackers are likely to be able to trigger the extraordinary codepath.
- ✓ **While the data in the paper** was unclear to the true impact of this attack, it could allow attackers who have found previously unexploitable bugs to revive them. It would be wise to consider the proposed mitigations if there are exposed applications that may be impacted by this class of exploitation enabler.

Figure 8. A figure showing how CHOP can bypass stack canaries and exploit a stack overflow.



Protect the System Call, Protect (Most of) the World with BASTION

Authors: Christopher Jelesnianski, Mohannad Ismail, Yeongjin Jang, Dan Williams, and Changwoo Min



This work explored a limited mitigation to limit system calls made by programs to only those that are calling an allowed system call, from the right location (with a correct call stack), with unmolested arguments. Their rationale is that most attackers aim to complete their exploitation with a system call to continue the attack. By focusing on that as a choke point, a variety of attacks can be mitigated (with less performance overhead than more broad mitigations against earlier stages of exploitation).

The authors introduce BASTION, an innovative approach that includes a compiler pass capable of analysing the control-flow graph to determine a valid chain of function calls for each system call, along with instrumentation on all data variables that impact the arguments of the system call. This is achieved through the integration of a shadow memory region with instrumented updates that are added to the program, resulting in enhanced functionality and security.

At runtime, a monitor verifies on a system call:

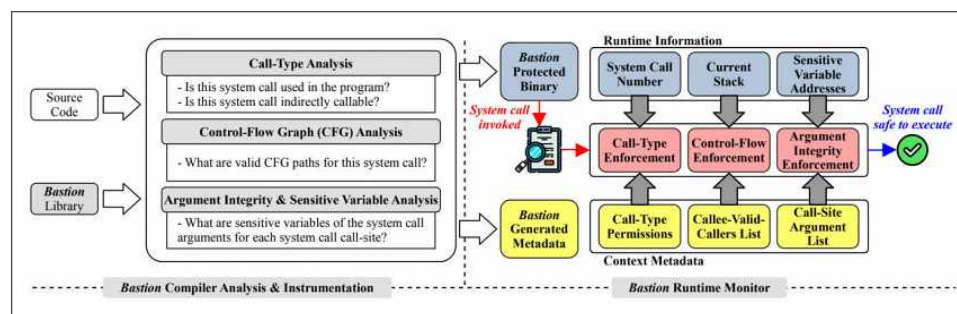
- the call is one that the program makes,
- it's been made from a valid chain of calls, and
- none of the arguments have been tampered with.

The performance overhead of BASTION was tested on nginx, SQLite, and vsFTPd; at worst there was a 2% slowdown.

TAKEAWAYS:

- ✓ **There are always counter-examples and edge cases** that are used to prevent adopting new mitigations, however in the interest of not letting perfect be the enemy of good, it is worth stepping back to see if the benefits outweigh the [modest] performance costs as well as the cost of recompilation and update. While this tool is compelling, the presence of many software packages and libraries that do not support the basics do not inspire hope of rapid adoption across the ecosystem.
- ✓ **While there are numerous attack classes** this work would not protect against, the idea of focusing more expensive mitigations on the most sensitive operations is compelling. For example, control flow integrity can be expensive if deployed broadly, yet if there are specific functions that should only be called correctly, it is less costly to deploy it in a more limited fashion.

Figure 9. A high-level diagram of the two components of BASTION and how they work to protect system calls.



3. Stepping back to gain perspective

- ✓ *Interoperability in End-to-End Encrypted Messaging*
- ✓ *High Risk Users and Where to Find Them*
- ✓ *Why I write my own security tooling*

Interoperability in End-to-End Encrypted Messaging

Authors: Esha Ghosh,
Paul Grubbs, Julia Len,
and Paul Rösler

-  [Slides](#)
-  [Paper](#)
-  [Video](#)

Figure 10. A timeline of the EU's DMA and when certain regulatory enforcements go into effect.

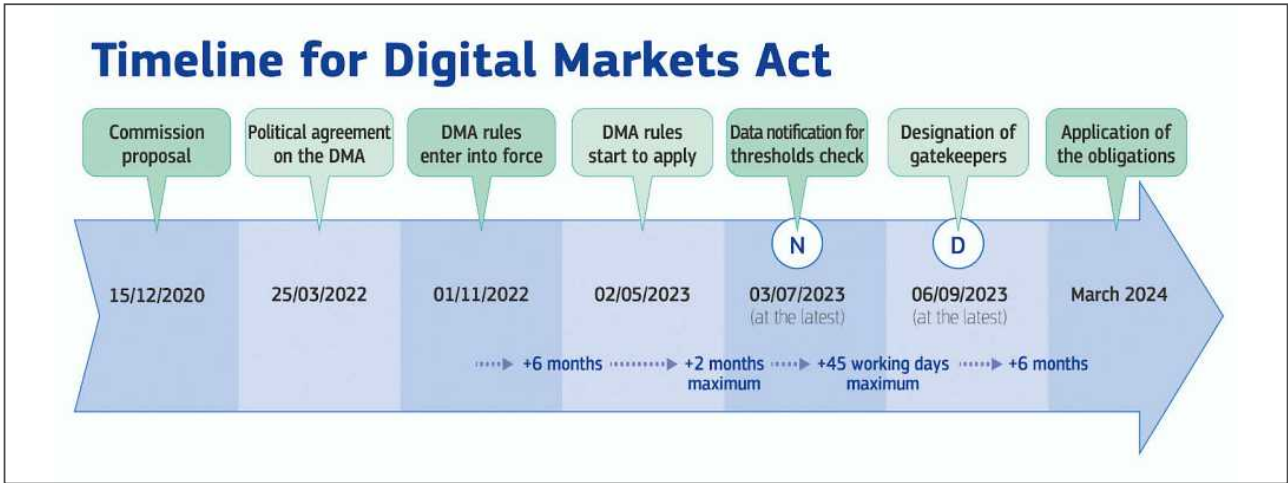


This paper walks through the European Union's new Digital Markets Act (DMA), which aims to limit network effects from large services. Of special interest are end-to-end encrypted (E2EE) messaging applications, such as WhatsApp or iMessage. In the middle of 2024, the EU will mandate that these services will interoperate without a loss of security (and over time provide more functionality). As there is limited time for this transition, and a few technical challenges, this work attempts to explore the options for how to manage disparate identity systems, E2EE protocols, and how abuse reports are handled.

For each challenge examined, the researchers explored different technical solutions and how it would impact security and privacy, for example, in a model where the server performs the translation between protocols, the E2EE guarantees are invalidated as the server will need to have sufficient key material to decrypt and re-encrypt the messages.

TAKEAWAYS:

- ✓ **The EU is stepping into a new role with the DMA** – dictating security and interoperability to target network effects. This new regulatory environment will be interesting to watch, and we will be keen to see how other services are impacted by this and future regulations.
- ✓ **The timeline on this rollout** is relatively small, so expect some bumps and finger-pointing when the inevitable issues arise.



High Risk Users and Where to Find Them

Author: Masha Sedova



[Paper](#)



[Video](#)

By analysing over 300,000 employees across various organisations, the researchers were able to find a statistically significant minority of users who were responsible for an outsized number of risky behaviours. They found that about 12% of users in an organisation accounted for between 30-50% of high-risk behaviours, such as clicking phishing emails, getting their system compromised by malware, or attempting to access sites identified as used to host C&C or drive-by downloads.

While the results don't stand out as too surprising (e.g., customer service departments more prone to clicking on suspicious emails, or R&D departments installing more untrusted software/browsing non-standard websites), it is good to see data supporting these hypotheses to allow for a more targeted response.

TAKEAWAY:

- ✓ **By matching defensive capabilities with expected risks,** a more efficient and targeted defence can be achieved, for example, ephemeral browsing and software installation environments for R&D teams, or stronger mail/account isolation for those handling external email inquiries.

Figure 11. A
breakdown of high-risk user behaviours by department.



	Malware	High Risk Category		
		Secure Browsing	Real Phishing	Simulated Phishing
Accounting/Finance	0.59%	12.26%	1.09%	10.09%
Admin/Ops	0.46%	13.83%	1.3%	9.41%
Board	0%		0%	9.52%
Creative	0%		0%	14.29%
Customer Service	0%	10.9%	0.09%	19.25%
Data Analysis	0.68%	13.33%	0%	16.55%
Engineering	0.58%	17.63%	0.44%	10.82%
Executive	0%	2.02%	0.37%	6.86%
HR	0.75%	16.43%	1.91%	12.08%
IT	0.24%	17.17%	0.75%	8.66%
Legal	0%	16.86%	1.82%	13.07%
Marketing	3.28%	14.29%	1.7%	15.93%
Medical Lab	0%	21.31%	3.43%	13.45%
Other	0.18%	22.32%	0.94%	10.54%
Product	0%	11.64%	1.12%	0.66%
Purchasing	0%		0%	8.88%
QA	0%		0%	9.25%
R&D	1.99%	19.05%	2.16%	1.08%
Sales	0.16%	1.67%	2.4%	14.51%

Why I write my own security tooling

Author: James Forshaw



[Code](#)



[Video](#)

This talk looks at a number of reported vulnerabilities by the researcher, and steps back to show the development of custom helper scripts or the applications that led to those discoveries. The author makes a compelling case for writing, documenting and sharing tools to help augment your work, act as a memory aid and to help others learn. Coming back to a target seven years after looking at it previously, the researcher was able to use the documentation he put into his code to remember the nuances of a poorly-documented API, and continue his recent work without trying to rediscover that knowledge.

TAKEAWAYS:

- ✓ **This talk underscores the offensive mindset** of building tools to search for chinks in the armour of a target, and then more tools to augment the manual aspects of building a proof-of-concept.
- ✓ **Defenders should learn from this mindset:** it is not enough to simply rely on existing or pre-built tools. A curious mind and some basic scripting goes a long way in getting weaknesses addressed before they are exploited.

Figure 12. A figure highlighting why the move to PQC is important to get started on now.





A view of Map of Africa outside Wilderness, South Africa.

4. Nifty sundries

- ✓ *Polynonce: A tale of a novel ECDSA attack and Bitcoin tears*
- ✓ *Finding 10x+ Performance Improvements in C++ with CodeQL*
- ✓ *Bridging the gap in the static and dynamic analysis of binaries through decompiler tomfoolery!*

Polynonce: A tale of a novel ECDSA attack and Bitcoin tears

Authors: Marco Macchetti and Nils Amiet

 [Blog](#)

 [Paper](#)

 [Code](#)

Figure 13. A log-scale chart showing the distribution of private keys used to sign Bitcoin transactions. While the majority of addresses are ephemeral and never reused, some clients reuse the same private key to sign transactions multiple times.



This research broadened a weakness in the ECDSA digital signature algorithm where if a random nonce was reused, the private key could be recovered. The researchers improved on that where if there were at least four signatures from the same private key, and a weakness in the nonce generation scheme (a bad cryptographic pseudo-random number generator, or CPRNG), the private key could be recovered as well. It is noted that in NIST's SP 800-22 document, an appendix included a number of weak PRNG algorithms as reference (since updated).

The researchers then explored two large public datasets of ECDSA signatures: the Bitcoin and Ethereum blockchains. After a lengthy process of downloading the entire chain, they created a dataset, grouping signatures by private key (see Figure). The dataset had many reused private key signatures, so they then ran their algorithm over the possible targets. While they did not find any wallets vulnerable to this stronger ECDSA weakness, they did find a number of wallets that had reused a nonce. That flaw was exploited by another party (or parties) and a total of almost 500 BTC was stolen.

TAKEAWAY:

- ✔ **This evaluation shows how something** that should never be found in the wild (nonce reuse or weak CPRNGs) finds a way to escape and have real-world impact. The inclusion of a weak PRNG in standards documentation is something to be avoided, much like an example code snippet posted without security – it will get copied into a production system.



Finding 10x+ Performance Improvements in C++ with CodeQL

Author: Sean Heelan



[Blog](#)



[Code](#)

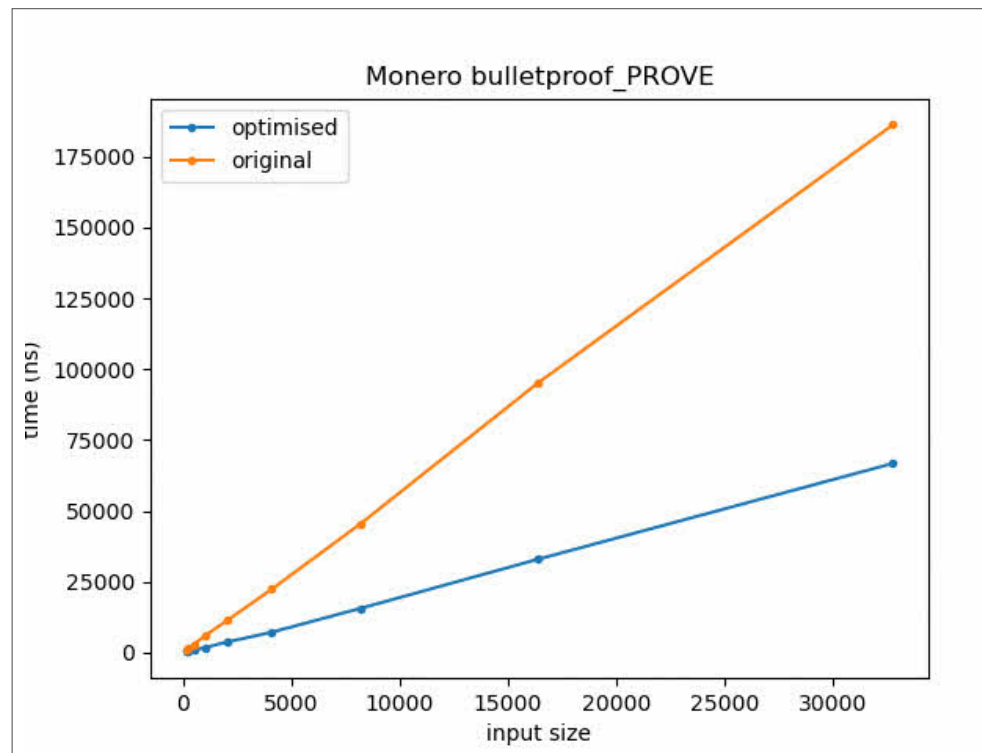
This work, by a well-known security researcher, focused on how to use a security static-analysis tool (CodeQL) to find code sections that could be optimised. The initial performance impact stems from a compiler aliasing issue, which adds extra memory accesses to verify that a distinct write hasn't changed a value that must be re-read. This led to the formalisation of some CodeQL queries that would look for this issue at a larger scale or in slightly different scenarios but with the same relevant properties.

The fix for the issue being explored was to simply change the type to one that was the same memory size, but wasn't treated as broadly aliasable (i.e., [unsigned] char to char8_t). In searching across large open-source C++ codebases, the author notes that the results from finding this pattern in loops was more impactful (since the code would be executed multiple times), and focused on those CodeQL results. Both Bitcoin and Monero codebases had instances of these performance issues, and were optimised with minor changes, resulting in significant performance improvements.

TAKEAWAYS:

- ✓ **The author's approach** of starting with the simplest and most concrete query for the SAST tool to verify assumptions, then working to generalise, is valid for security practitioners as well as those working to optimise code.
- ✓ **Tools like CodeQL** make it easy to scale your queries, but that can also dramatically scale the false positives or edge cases.

Figure 14. A graph showing the performance time of the optimised versus unoptimised code in Monero's `bulletproof_PROVE` function. For realistic input sizes there's a 3-4x speedup for changing a single type assignment.



Bridging the gap in the static and dynamic analysis of binaries through decompiler tomfoolery!

Author: Zion Basque



[Code](#)



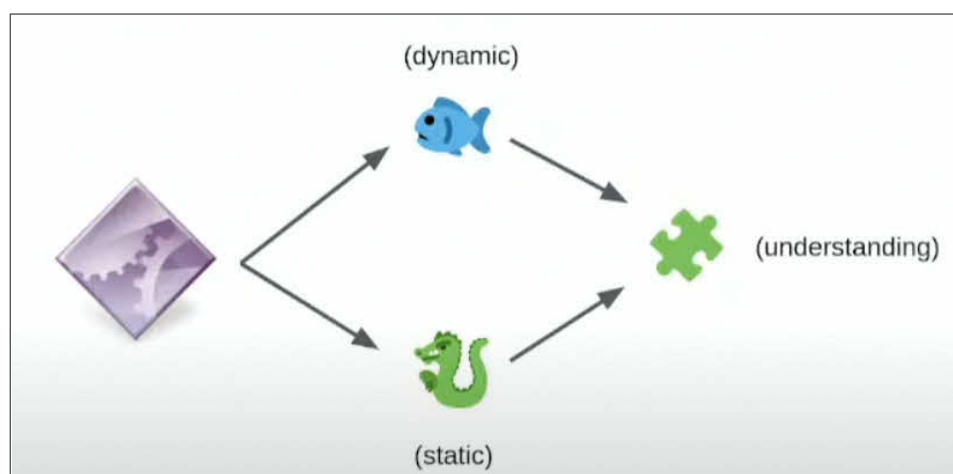
[Video](#)

This work explored a toolchain to automate information exchanges between static reverse engineering tooling (i.e., decompilers) with a dynamic debugging environment. Integrating these two [typically] distinct stages of reverse engineering and vulnerability research into a single cohesive tool can aid in overall target understanding. By interleaving decompiled code, and generated symbols with a live single-stepping debugger, a more information-rich experience can be provided. The talk ended with a live demo of remotely debugging a wireless router and a new exploit discovered with this toolchain.

TAKEAWAY:

- ✓ **Context switching** is expensive in human processes just as in software; tooling like decomp2dbg helps reduce that friction, and has shown its ability to pay off already.

Figure 15. A high-level diagram of how there are primarily two separate paths towards understanding a binary; decomp2dbg helps combine these two paths.



Conclusion

Even with many more specialised and topic-focused conferences this quarter, we were able to find a number of broadly impactful works to highlight.

THREE THEMES WERE HIGHLIGHTED THIS QUARTER:

1. Modern security techniques for modern environments.
2. Novel attack classes and defences/mitigations.
3. A step back from purely technical research to gain perspective.

Next quarter should see a quantity increase as we move out of winter in the northern hemisphere. We look forward to it!

